

LAB-2

PostgreSQL 18.6 Manual Failover and Rebuild

DB1 Failure → Promote DB2 → Rebuild DB1 as Standby

AWS EC2 | Amazon Linux 2023 | Native Physical Streaming Replication

Document	Value
Lab	LAB-2
Platform	AWS EC2
Database	PostgreSQL 18.6
Operating System	Amazon Linux 2023
Replication	Native Physical Streaming Replication - Asynchronous
Tested by	Praveen Madupu
Test date	02 September, 2026 - Wednesday
Status	Tested and Successfully Validated

Training-lab notice: The credentials shown in this SOP are intentionally retained because this is a non-production training environment. Production environments should use a secrets-management solution and restricted network access.

LAB-1:

https://github.com/PMSQLDBA/PostgreSQL_Admin_Articles/blob/main/Lab-1_PostgreSQL18_AWS_EC2_Physical_Streaming_Replication_SOP-02Sep2026-Wednesday.pdf

Table of Contents

Section	Page
1. Purpose and Scope	3
2. Lab Objective and Final Result	3
3. Lab Architecture	3
4. Environment Inventory	4
5. Preconditions and Safety Controls	4
6. Validate Replication Before Failover	5
7. Create Pre-Failure Data Marker	5
8. Simulate DB1 Failure / Fence	5
9. Validate and Promote DB2	6
10. Post-Promotion Validation Row	6
11. Prepare DB2 for DB1 Standby	6
12. Prepare DB1 Authentication	7
13. Validate Replication Protocol	7
14. Preserve Former DB1 PGDATA	8
15. Rebuild DB1 with pg_basebackup	8
16. Validate/Clean primary_conninfo	8
17. Start DB1 as New Standby	9
18. Validate DB1 WAL Receiver	9
19. Validate DB2 WAL Sender/Lag	10
20. Validate Data After Rebuild	10
21. LAB-2 Acceptance Criteria	11
22. Final LAB-2 State	11
23. Operational and Cleanup Notes	11
24. Quick Validation Command Set	12
25. Official Documentation References	12
26. Lab Completion Sign-off	13

1. Purpose and Scope

This SOP documents the completed LAB-2 exercise for manual PostgreSQL failover and rejoin on the two-node PostgreSQL 18.6 AWS EC2 lab created in LAB-1. The test intentionally stopped DB1, promoted DB2 to the new writable primary, preserved the old DB1 data directory, rebuilt DB1 from a fresh physical base backup taken from DB2, and validated the reversed streaming-replication direction.

The procedure below reflects the successfully tested path. Historical command mistakes and transient connection failures are not included as procedural steps.

Important terminology: After DB2 is promoted and DB1 is rebuilt from DB2, the resulting topology is DB2 Primary → DB1 Standby. This is a failover plus rejoin/rebuild. Returning the roles to DB1 Primary → DB2 Standby would require a separate controlled switchover/failback exercise.

2. Lab Objective and Final Result

- Simulate a DB1 primary-server failure by stopping PostgreSQL on DB1.
- Confirm DB2 is still a standby before promotion.
- Promote DB2 manually with `pg_promote()`.
- Validate that DB2 becomes writable and leaves recovery mode.
- Keep the old DB1 fenced to avoid split-brain.
- Allow DB1 to connect to DB2 using the physical replication role.
- Preserve the former DB1 PGDATA for rollback/reference.
- Rebuild DB1 using `pg_basebackup` from the promoted DB2 primary.
- Start DB1 as a standby and validate WAL streaming from DB2.
- Validate end-to-end data replication, read-only status, and zero-byte replay lag at the final check.

Final outcome: LAB-2 completed successfully. DB2 is the active read/write Primary and DB1 is the read-only Standby receiving asynchronous physical WAL streaming from DB2.

3. Lab Architecture

3.1 Starting topology from LAB-1

```

DB1 PRIMARY
172.30.2.177
PostgreSQL 18.6
Read / Write
|
| Asynchronous Physical Streaming Replication
v
DB2 HOT STANDBY
172.30.2.128
PostgreSQL 18.6
Read Only

```

3.2 Final topology after LAB-2

```

DB2 PRIMARY
172.30.2.128
PostgreSQL 18.6
Read / Write
|
| Asynchronous Physical Streaming Replication
| Final observed lag: 0 bytes
v
DB1 STANDBY
172.30.2.177
PostgreSQL 18.6
Read Only

```

4. Environment Inventory

Item	DB1	DB2
EC2 Name	pgsqldb1	pgsqldb2
Hostname	db1	db2
Private IPv4	172.30.2.177	172.30.2.128
Public IPv4	3.15.171.222	18.218.173.106
Operating System	Amazon Linux 2023	Amazon Linux 2023
PostgreSQL	18.6	18.6
PGDATA	/var/lib/pgsql/data	/var/lib/pgsql/data
Port	5432	5432
Initial role	Primary	Standby
Final LAB-2 role	Standby	Primary

Lab credential	Value
Database	labdb
Application role	labuser
Application password	PgLab@2026
Replication role	replicator
Replication password	ReplLab@2026

5. Preconditions and Safety Controls

- LAB-1 physical streaming replication must already be healthy before starting the failover test.
- DB1 and DB2 must resolve each other through /etc/hosts or DNS.
- TCP 5432 must be permitted between the two EC2 instances.
- The replication role replicator must exist and have LOGIN and REPLICATION privileges.
- After DB2 is promoted, DB1 must remain stopped until it has been safely rebuilt or otherwise reconciled with the new primary timeline.
- Do not start the former DB1 writable cluster after DB2 promotion. Running both writable nodes independently would create split-brain risk.
- The test uses no automatic failover manager, no permanent physical replication slot, and no WAL archive.

6. Step 1 - Validate Replication Before Failover

6.1 Check DB1 as the current primary

```
hostname
hostname -I
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_timestamp AS check_time;
"
```

Validated result in this lab: DB1 was active and `pg_is_in_recovery()` returned f.

```
sudo -u postgres psql -c "
SELECT
    client_addr AS standby_ip,
    state,
    sync_state AS mode,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn,replay_lsn)) AS lag
FROM pg_stat_replication;
"
```

Validated result: DB2 172.30.2.128 was streaming asynchronously with 0 bytes lag.

7. Step 2 - Create a Pre-Failure Data Marker on DB1

```
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-2: Before DB1 Failure');
"

sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id;
"
```

Validated test row: ID 3 - LAB-2: Before DB1 Failure.

8. Step 3 - Simulate DB1 Failure and Fence the Old Primary

```
sudo systemctl stop postgresql
sudo systemctl is-active postgresql
sudo ss -lntp | grep 5432 || echo "DB1 PostgreSQL port 5432 is DOWN"
```

Validated result: PostgreSQL on DB1 was inactive and TCP 5432 was no longer listening.

Safety control: From this point forward, DB1 must remain stopped until it is rebuilt as a standby. This is the most important split-brain prevention control in this manual lab.

9. Step 4 - Validate DB2 and Promote It to Primary

9.1 Validate DB2 before promotion

```
sudo systemctl is-active postgresql
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"

```

Before promotion, DB2 remained in recovery mode.

9.2 Promote DB2

```
sudo -u postgres psql -c "
SELECT pg_promote();
"
```

Validated result: pg_promote() returned t.

9.3 Validate DB2 after promotion

```
sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    current_setting('transaction_read_only') AS read_only;
"
```

Validated result: is_standby = f and read_only = off. DB2 was now the writable Primary.

10. Step 5 - Create a Post-Promotion Validation Row on DB2

```
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-2: DB2 Promoted to New Primary');
"

sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id;
"
```

Validated row: ID 36 - LAB-2: DB2 Promoted to New Primary.

Sequence note: The identity value jumped from 3 to 36. PostgreSQL sequence/identity values are not guaranteed to be gapless. A gap does not indicate a replication problem.

11. Step 6 - Prepare DB2 to Accept DB1 as a New Standby

11.1 Verify the replication role on DB2

```
sudo -u postgres psql -c "
SELECT rolname, rolcanlogin, rolreplication
FROM pg_roles
WHERE rolname = 'replicator';
"
```

Validated result: replicator existed with LOGIN = true and REPLICATION = true.

11.2 Add the DB1 replication HBA rule

```
sudo grep -n "172.30.2.177" /var/lib/pgsql/data/pg_hba.conf

echo "host    replication    replicator    172.30.2.177/32    scram-sha-256" | \
sudo tee -a /var/lib/pgsql/data/pg_hba.conf
```

Validated rule: host replication replicator 172.30.2.177/32 scram-sha-256.

11.3 Validate and reload HBA

```
sudo -u postgres psql -c "
SELECT line_number, type, database, user_name, address, auth_method, error
FROM pg_hba_file_rules
WHERE error IS NOT NULL;
"

sudo systemctl reload postgresql
sudo systemctl is-active postgresql
```

Validated result: 0 HBA errors and DB2 remained active.

12. Step 7 - Prepare DB1 Replication Authentication

```
hostname
hostname -I
sudo systemctl is-active postgresql
sudo ss -lntp | grep 5432 || echo "DB1 PostgreSQL is safely stopped"
```

Validated result: hostname db1, private IP 172.30.2.177, PostgreSQL inactive.

```
sudo bash -c 'cat > /var/lib/pgsql/.pgpass <<EOF
db2:5432:replication:replicator:ReplLab@2026
172.30.2.128:5432:replication:replicator:ReplLab@2026
EOF'

sudo chown postgres:postgres /var/lib/pgsql/.pgpass
sudo chmod 600 /var/lib/pgsql/.pgpass
sudo ls -l /var/lib/pgsql/.pgpass
getent hosts db2
```

Validated permissions: -rw----- owned by postgres:postgres. Name resolution returned 172.30.2.128 for db2.

13. Step 8 - Validate the Physical Replication Protocol from DB1 to DB2

```
sudo -u postgres psql \
"host=db2 port=5432 user=replicator replication=true" \
-c "IDENTIFY_SYSTEM;"
```

Validated output:

```
systemid          : 7680977547376578577
timeline          : 2
xlogpos           : 0/4040208
```

Why timeline 2 matters: DB2 created a new timeline when it was promoted. Confirming timeline 2 verifies that DB1 is connecting to the promoted DB2 history rather than the old DB1 timeline.

14. Step 9 - Preserve the Former DB1 PGDATA

```
sudo systemctl is-active postgresql

sudo mv /var/lib/pgsql/data \
/var/lib/pgsql/data_old_primary_lab2_20260902

sudo mkdir -p /var/lib/pgsql/data
sudo chown postgres:postgres /var/lib/pgsql/data
sudo chmod 700 /var/lib/pgsql/data

sudo ls -ld /var/lib/pgsql/data
sudo ls -ld /var/lib/pgsql/data_old_primary_lab2_20260902
sudo ls -la /var/lib/pgsql/data
```

Validated state: DB1 remained inactive; the old primary PGDATA was preserved; the replacement PGDATA was empty and owned by postgres with mode 700.

15. Step 10 - Rebuild DB1 with pg_basebackup from DB2

```
sudo -u postgres pg_basebackup \
-h db2 \
-p 5432 \
-U replicator \
-D /var/lib/pgsql/data \
-Fp \
-Xs \
-P \
-R
```

Validated completion: 32397/32397 kB (100%), 1/1 tablespace.

Meaning of the options: -Fp uses plain-format files, -Xs streams required WAL during the backup, -P shows progress, and -R creates standby.signal plus the standby connection settings for the source server.

16. Step 11 - Validate Standby Files and Clean the Inherited primary_conninfo

```
sudo ls -l /var/lib/pgsql/data/standby.signal
sudo cat /var/lib/pgsql/data/PG_VERSION
sudo grep -n "primary_conninfo" /var/lib/pgsql/data/postgresql.auto.conf
```

Validated: standby.signal existed and PG_VERSION was 18. The physical copy inherited DB2's old primary_conninfo for db1, while pg_basebackup -R added a new primary_conninfo for db2.

Configuration cleanup: Only the new upstream DB2 entry should remain in DB1 postgresql.auto.conf. The stale inherited host=db1 entry was removed before starting DB1.

```
sudo cp /var/lib/pgsql/data/postgresql.auto.conf \
/var/lib/pgsql/data/postgresql.auto.conf.before_lab2_cleanup

sudo chown postgres:postgres \
/var/lib/pgsql/data/postgresql.auto.conf.before_lab2_cleanup

sudo sed -i "/primary_conninfo.*host=db1 /d" \
/var/lib/pgsql/data/postgresql.auto.conf

sudo grep -n "primary_conninfo" \
/var/lib/pgsql/data/postgresql.auto.conf
```

Final validated entry pointed to host=db2, port=5432, user=replicator.

17. Step 12 - Start DB1 as the New Standby

```
sudo systemctl start postgresql
sudo systemctl is-active postgresql

sudo -u postgres psql -c "
SELECT
    pg_is_in_recovery() AS is_standby,
    pg_last_wal_receive_lsn() AS receive_lsn,
    pg_last_wal_replay_lsn() AS replay_lsn;
"
```

Validated result: DB1 became active, pg_is_in_recovery() returned t, and receive/replay LSNs matched at 0/6000168.

18. Step 13 - Validate DB1 WAL Receiver

```
sudo -u postgres psql -c "
SELECT
    status,
    sender_host,
    sender_port,
    latest_end_lsn,
    slot_name
FROM pg_stat_wal_receiver;
"
```

Validated result: status = streaming, sender_host = db2, sender_port = 5432, latest_end_lsn = 0/6000168. No permanent replication slot was used in this lab.

19. Step 14 - Validate DB2 WAL Sender and Replication Lag

```
sudo -u postgres psql -c "
SELECT
    application_name,
    client_addr,
    state,
    sync_state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn,replay_lsn)) AS replication_lag
FROM pg_stat_replication;
"
```

Validated result on DB2: client_addr 172.30.2.177, state streaming, sync_state async, and replication_lag 0 bytes.

20. Step 15 - Validate Data After Rebuild

20.1 Confirm the post-promotion row exists on DB1

```
sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id;
"
```

DB1 displayed the row ID 36 - LAB-2: DB2 Promoted to New Primary, proving the rebuilt standby contains data generated on DB2 after promotion.

20.2 Confirm DB1 is read-only

```
sudo -u postgres psql -d labdb -c "
SHOW transaction_read_only;
"
```

Validated result: on.

20.3 Insert one more row on DB2 and confirm it reaches DB1

```
# Run on DB2 Primary
sudo -u postgres psql -d labdb -c "
INSERT INTO replication_validation(message)
VALUES ('LAB-2: DB2 Primary to DB1 Standby Validation');
"

# Run on DB1 Standby
sudo -u postgres psql -d labdb -c "
SELECT id, message, created_at
FROM replication_validation
ORDER BY id DESC
LIMIT 5;
"
```

Validated result on DB1: ID 37 - LAB-2: DB2 Primary to DB1 Standby Validation.

21. LAB-2 Acceptance Criteria

Validation	Expected / Tested Result	Status
DB1 stopped before DB2 promotion	inactive; TCP 5432 down	PASS
DB2 promotion	pg_promote() returned true	PASS
DB2 role after promotion	pg_is_in_recovery() = f	PASS
DB2 write mode	transaction_read_only = off	PASS
Replication login	IDENTIFY_SYSTEM successful; timeline 2	PASS
Old DB1 PGDATA preserved	data_old_primary_lab2_20260902 exists	PASS
DB1 base backup	100% complete	PASS
DB1 standby mode	pg_is_in_recovery() = t	PASS
DB1 WAL receiver	streaming from db2:5432	PASS
DB2 WAL sender	streaming to 172.30.2.177	PASS
Replication mode	async	PASS
Observed final lag	0 bytes	PASS
Post-promotion row visible on DB1	ID 36 present	PASS
Final DB2→DB1 row visible on DB1	ID 37 present	PASS
DB1 read-only status	transaction_read_only = on	PASS

22. Final LAB-2 State

Server	Role	Recovery	Read-only	Replication view
DB2 - 172.30.2.128	Primary	f	off	pg_stat_replication shows DB1
DB1 - 172.30.2.177	Standby	t	on	pg_stat_wal_receiver shows db2

```
DB2 PRIMARY (172.30.2.128)
pg_is_in_recovery() = f
transaction_read_only = off
pg_stat_replication = DB1 streaming / async / 0 bytes

WAL →

DB1 STANDBY (172.30.2.177)
pg_is_in_recovery() = t
transaction_read_only = on
pg_stat_wal_receiver = streaming from db2
```

23. Operational and Cleanup Notes

- Retain /var/lib/postgresql/data_old_primary_lab2_20260902 temporarily as rollback/reference evidence until LAB-2 documentation and screenshots are finalized.

- Do not attempt another `pg_basebackup` into the active DB2 `/var/lib/pgsql/data` directory. DB2 is the live Primary after LAB-2.
- The blank `inet_server_addr()` seen when connecting locally through the Unix-domain socket is normal. Use a TCP connection if the server IP must be shown by `inet_server_addr()`.
- Sequence/identity values can contain gaps. Do not use sequence continuity as a replication-health indicator.
- This lab intentionally uses asynchronous replication. A clean zero-byte lag check is a point-in-time observation, not a guarantee that no transaction can be lost during a future asynchronous failover.
- For production HA, use a tested failover-management design, explicit fencing, monitoring, WAL-retention/archiving strategy, backup policy, and credential management.

24. Quick Validation Command Set

24.1 DB2 Primary

```
sudo -u postgres psql -c "
SELECT pg_is_in_recovery(), current_setting('transaction_read_only');
"

sudo -u postgres psql -c "
SELECT client_addr, state, sync_state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag
FROM pg_stat_replication;
"
```

24.2 DB1 Standby

```
sudo -u postgres psql -c "
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn();
"

sudo -u postgres psql -c "
SELECT status, sender_host, sender_port, latest_end_lsn
FROM pg_stat_wal_receiver;
"

sudo -u postgres psql -d labdb -c "SHOW transaction_read_only;"
```

25. PostgreSQL Official Documentation References

- **Warm Standby / Failover:** <https://www.postgresql.org/docs/18/warm-standby-failover.html>
- **High Availability, Load Balancing, and Replication:** <https://www.postgresql.org/docs/18/high-availability.html>
- **Hot Standby:** <https://www.postgresql.org/docs/18/hot-standby.html>
- **Server Administration Functions - `pg_promote`:** <https://www.postgresql.org/docs/18/functions-admin.html>
- **Replication Configuration:** <https://www.postgresql.org/docs/18/runtime-config-replication.html>
- **`pg_basebackup`:** <https://www.postgresql.org/docs/18/app-pgbasebackup.html>
- **The `pg_hba.conf` File:** <https://www.postgresql.org/docs/18/auth-pg-hba-conf.html>
- **Monitoring Database Activity / Replication Views:** <https://www.postgresql.org/docs/18/monitoring-stats.html>

26. Lab Completion Sign-off

Field	Value
Lab	LAB-2 - Manual Failover and Rebuild
Tested by	Praveen Madupu
Date	02 September, 2026 - Wednesday
Result	PASS - Successfully completed
Final topology	DB2 Primary → DB1 Standby
Replication	Physical streaming, asynchronous
Observed final replication lag	0 bytes

LAB-2 status: Manual failover to DB2, DB1 rebuild, role reversal, and end-to-end replication validation completed successfully.